Validator-Guided Repair on a Shared Initial LLM Candidate: A Preregistered Paired Evaluation for Network Configuration Safety

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory experiment (PREREG-CAND-001-VER-GVR-2026-09-01) to measure whether permitting a deterministic network-configuration validator plus one automated repair changes the rate of validator-valid executable configurations relative to leaving the identical sampled LLM candidate unguarded (shared_initial_candidate semantics). Using the declared generator gpt-5-mini (hosted adapter openai_responses) and deterministic_netops_validator_v5 on N = 200 paired intents (completed_episode_count = 400; model_calls_used = 200), every shared initial candidate passed validator checks and no repairs were attempted or applied (repair_attempt_count = 0; repair_applied_count = 0). Observed outcomes: baseline_success = 200/200, guarded_success = 200/200; paired contingency n11 = 200, n10 = 0, n01 = 0, n00 = 0; exact McNemar two-sided p = 1.0; paired bootstrap percentile 95% CI (seed = 1729; 10,000 resamples) = [0.0, 0.0]. We present artifact-grounded methodology, execution accounting, representative validator diagnostics, compact contingency and repair summaries, and a focused discussion clarifying the causal limits of the executed estimand under shared_initial_candidate semantics and preregistered follow-on designs required to measure repair efficacy under independent sampling, higher difficulty, or adversarial inputs.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration sequences in network operations (NetOps). Erroneous sequences, syntactic mistakes, or fabricated fields can cause service disruption or policy violations when applied to devices. A common architectural mitigation is a generate→validate→repair (GVR) pipeline: generate candidate configuration(s), deterministically validate them against intent/topology/workflow constraints, and trigger automated repair when the validator finds faults. Prior work documents verifier-guided improvements in infrastructure-as-code and related code-generation domains and recommends grounding strategies to reduce hallucination in operational tasks [1–3]. However, whether verifier-guided pipelines reduce execution-level operational risk for network configurations remains an open empirical question because prior demonstrations often measure textual correctness or IaC test suites rather than executed device-state safety in packet-level or emulator contexts [3,4].

This paper reports a fully preregistered paired confirmatory experiment that isolates the downstream causal contribution of a deterministic validator and any permitted repair acting on the exact same sampled LLM candidate (shared_initial_candidate semantics). Under these semantics baseline and guarded episodes for each intent begin from the identical initial generation; any paired difference can therefore only arise from deterministic validation and a permitted repair of that same candidate. Our primary research question is: under shared_initial_candidate semantics, does permitting a deterministic validator plus at most one automated repair change the proportion of validator-valid executable configurations relative to leaving the same initial candidate unguarded? The contribution is an artifact-grounded report of the executed estimand with complete execution accounting, exact inferential statistics, representative validator diagnostics, and precise recommendations for preregistered follow-on designs that can measure repair efficacy when independent sampling, elevated difficulty, or adversarial inputs are employed.

II. Related Work

Hallucination and factual unreliability in LLM outputs are well documented and motivate grounding, retrieval, and verification strategies for operational tasks [1]. Retrieval-augmented and evidence-grounded pipelines can reduce hallucination in operations-style QA, though these works evaluate textual or documentation correctness rather than device-level executability [2]. Generate→verify→repair architectures have shown empirical improvements in IaC and program generation (for example, verifier-guided fine-tuning and policy-guided repair loops) and are a primary methodological inspiration for our work [3]. Recent NetOps evaluations examine intent translation and LLM capability but emphasize capability benchmarks rather than standardized execution-level safety measures that map generated commands to executable device states in a simulator/emulator [4]. Comprehensive AIOps surveys highlight the need for execution-aware benchmarks and validator fidelity uplift to bridge generation to operational safety [5].

From these verified sources we identified two methodological gaps that motivated the current paired design. First, verifier-guided demonstrations in adjacent domains do not directly measure whether corrections prevent unsafe executable states when enacted on networks; empirical transferability to NetOps therefore requires execution-level validation. Second, many prior studies mix sampling variability with downstream validator/repair effects; precisely defined estimands that separate these sources of variation are uncommon. We therefore preregistered a shared_initial_candidate paired design that isolates the downstream validator/repair effect on identical generator outputs.

III. Methodology

Preregistration and primary estimand. The study preregistration (PREREG-CAND-001-VER-GVR-2026-09-01) froze the primary estimand and analysis before any model calls. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (per-intent paired difference in proportion of validator-valid executable configurations) under shared_initial_candidate semantics. Design: deterministic paired sample of task_count = 200 intents producing planned_episode_count = 400 episodes. Generation semantics: exactly one sampled initial generation per pair (shared) and deterministic reuse by baseline and guarded episodes. Guarded episodes allowed up to one automated repair (maximum_repair_calls_per_task = 1). Primary test: exact two-sided McNemar test; paired bootstrap (seed = 1729; 10,000 resamples) for a percentile 95% CI.

Task bank and contamination controls. A deterministic task generator produced the manifest of 200 synthetic intent_configuration_repair_v1 tasks. Each manifest entry records initial_state, required_sequence, difficulty annotations, and a generation_seed for deterministic replay. Preexecution contamination checks (SHA-256 exact-match and n-gram similarity thresholds) were applied; no exact-match contamination exclusions were flagged and the confirmatory sample executed intact. Contamination and task manifest artifacts are archived in the execution evidence bundle.

Model calls, generation semantics, and provider audit. Execution used the declared generator labeled gpt-5-mini via the hosted adapter openai_responses and consumed model_calls_used = 200 (one shared initial generation per pair). The archived model_configuration.json records declared_model_version = "gpt-5-mini" and temperature = null/unset; we explicitly state that null/unset is not evidence of deterministic sampling and does not imply temperature = 0. Provider audit traces record hosted_model_call_event_count = 200, cache_miss_count = 200, and stage_counts showing shared_initial_generation = 200 consistent with the preregistered shared_initial_candidate semantics.

Deterministic validator and repair policy. Scoring used deterministic_netops_validator_v5 with scoring_rule = "full_intent_constraint_validation." The validator parses candidate commands, applies task workflow_policies and required_sequence checks, and produces structured validator traces with validation_before and validation_after subtables, parsed_command_count, required_command_count, normalized_configuration, violation_codes, and a repair_applied flag. For guarded episodes one automated repair attempt was permitted; the repair, if invoked, would produce a new candidate which the validator would re-validate. Under shared_initial_candidate semantics only the guarded arm could alter the identical initial candidate via the permitted repair; the baseline arm reused the same candidate unchanged.

Mapping to outcomes and missingness. Validator.valid = true mapped to a binary "executable/valid" outcome for each episode. Primary analysis adhered to the preregistered complete_pair_primary policy (exclude incomplete pairs). A sensitivity analysis (failure-as-zero) was preregistered to treat missing or failed episodes as unsafe. Multiplicity: single confirmatory McNemar test; exploratory subgroup analyses (e.g., by difficulty patterns) were labeled exploratory and corrective procedures (Benjamini–Hochberg) were specified if applied.

IV. Results

Execution accounting and deterministic reconciliation. The execution completed exactly as preregistered: planned_episode_count = 400, completed_episode_count = 400, failed_episode_count = 0, and model_calls_used = 200 (one shared initial generation per pair). Analysis missingness_summary reports complete_pairs = 200 with zero failed or unscorable episodes. Deterministic reconciliation records hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, cache_miss_count = 200, and stage_counts = {"shared_initial_generation": 200}; cross_condition_cache_key_reuse_observed = false and all deterministic consistency checks passed.

Primary binary outcomes and contingency. The validator mapped each episode to a binary executable outcome (validator.valid). Condition summaries show baseline_successes = 200/200 and guarded_successes = 200/200 (success_rate = 1.0 for both arms). The paired 2×2 contingency table is n11 = 200, n10 = 0, n01 = 0, n00 = 0; exact McNemar two-sided p = 1.0. The paired bootstrap (seed = 1729; 10,000 resamples) produced a percentile 95% CI for the paired difference of [0.0, 0.0]. These files are archived (analysis/condition_summary.csv and analysis/paired_contingency_table.csv) and exactly reproduce the contingency and CI numbers reported here.

Repair activity, validator diagnostics, and representative examples. Across the confirmatory sample the validator flagged no violations on any shared initial candidate; as a result repair_attempt_count = 0 and repair_applied_count = 0. Representative per-episode scoring JSON artifacts illustrate the pattern. For example, task-000004 (pair_id pair-000004) shared_initial_candidate and both condition responses were identical:

interface edge2 admin down interface edge2 vlan 40 interface edge2 admin up interface edge1 admin down

For task-000004 the archived validator trace shows parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, and validation_after.valid = true. Similarly, task-000005 shows parsed_command_count = 3 and required_command_count = 3 with validation_after.valid = true; task-000006 shows parsed_command_count = 6 and required_command_count = 6 with validation_after.valid = true. These per-episode diagnostics (execution/scoring/task-00000X-*.json) demonstrate that the validator parsed the expected command set, matched required_sequence checks, and reported no violation codes for the sampled candidates. Because the validator never produced a violation, the guarded repair path was never invoked and could not alter any outcome.

Contamination and provider audit summary. Preexecution contamination checks found no exact-match contamination exclusions; analysis/contamination_summary.csv is empty. Provider audit and execution manifests corroborate the sampling semantics: hosted_model_call_event_count = 200, cache_miss_count = 200, and stage_counts indicating exactly one shared_initial_generation per pair. No infrastructure retries or failures occurred; analysis/analysis_log.jsonl documents the paired analysis completion and bootstrap parameters (bootstrap_resamples = 10000; bootstrap_seed = 1729).

Compact repair summary (explicit). In concise preregistered terms: total_pairs = 200; repair_attempt_count = 0; repair_applied_count = 0. The empty repair counts fully explain the degenerate paired contingency (all pairs valid before any repair opportunity). All artifacts supporting these counts are archived in the evidence bundle (notably the execution/scoring JSONs, execution/task_manifest.jsonl, analysis/paired_contingency_table.csv, and analysis/condition_summary.csv) and permit independent verification that no repairs were produced or applied in guarded episodes.

V. Discussion

Interpreting the executed estimand. The shared_initial_candidate semantics were intentionally chosen to isolate the causal effect of deterministic validation and any permitted repair acting on the exact same generator output. The executed estimand therefore answers a narrowly defined causal question: when baseline and guarded episodes begin from the identical sampled candidate, can deterministic validation and at most one automated repair change the validator-valid/executable outcome? The observed null effect (paired difference = 0.0) shows that, for this task bank and this single sampled candidate per pair, the validator found no violations and the permitted repair path was never exercised. Importantly, this does not evaluate whether guarded pipelines reduce first-pass generator error rates under independent sampling or different model temperatures; such claims would require a different preregistered estimand and execution.

Concrete validator behavior and why the guarded path was not invoked. The deterministic_netops_validator_v5 used in scoring enforces a structured sequence of checks: lexical/command parsing, mapping commands to normalized_configuration, counting parsed_command_count against required_command_count from the task manifest, and evaluating workflow_policies such as "minimum_active_in_groups" and "must_be_down_for_fields." The archived per-episode JSON traces show parsed_command_count == required_command_count and validation_after.valid == true for representative tasks (e.g., task-000004: parsed_command_count = 4, required_command_count = 4), which means the validator accepted the shared initial candidate as satisfying the syntactic and workflow constraints recorded in the task manifest. Because the validator reported zero violation_codes across the confirmatory sample, the guarded branch's single permitted automated repair had no trigger and therefore repair_attempt_count = 0 and repair_applied_count = 0. This artifact-level diagnostic fully explains the degenerate paired contingency (n11 = 200) observed in the results.

Statistical and power implications given zero discordance. The exact McNemar two-sided p = 1.0 and bootstrap CI = [0.0, 0.0] are deterministic consequences of n10 = n01 = 0. From an inferential perspective, zero discordant pairs carry no information about the sign or magnitude of a potential repair benefit under sampling regimes that would produce validator failures. The preregistered power analysis assumed nonzero baseline unsafe rates (sensitivity checks considered baseline unsafe rates in $\{0.2, 0.3, 0.4\}$ and target effects $\approx$ 0.08–0.12). Those assumptions remain necessary: to detect an absolute risk reduction on the order of 10 percentage points with ~80% power requires an observable rate of validator failures under the baseline sampling distribution. In practice this means either (a) selecting task items with a higher expected difficulty/failure label, (b) altering generation sampling to increase first-pass diversity (e.g., nonzero temperature), or (c) injecting adversarial perturbations so that the guarded repair path is exercised. Without such adjustments the paired shared_initial_candidate design can produce informative nulls when the validator accepts nearly all sampled candidates, as occurred here.

Construct validity of the validator abstraction. Deterministic_netops_validator_v5 provides strong syntactic and workflow-policy checks and records structured diagnostics that make its operation auditable in artifact traces. However, construct validity limits remain: the validator maps candidate commands to an asserted final_state and enforces required_sequence and policy constraints, but it does not execute vendor CLI on hardware nor perform packet-level emulation of run-time protocol interactions. As a result, certain dynamic failure modes (e.g., transient routing convergence, BGP session flaps, or timing-dependent failover behavior) are outside the validator's scope. The evidence bundle therefore supports a qualified conclusion only about validator-level syntactic/workflow correctness for the sampled candidates, not about runtime-observable failures that a live device or packet-level emulator might reveal. Representative task annotations in the manifest (difficulty levels labeled "very_hard" and an isolated "extreme" instance) indicate the generator was challenged by multi-step required_sequences, yet the validator still accepted the single sampled generations; this suggests that either the generator produced compliant plans for these synthetic tasks or that the task bank difficulty distribution did not produce the class of subtle errors that would need repair.

Operational implications and conservative interpretation. Practitioners evaluating GVR pipelines should note two operational lessons from these artifacts. First, an observed guarded==baseline null under shared_initial_candidate semantics does not imply that a GVR pipeline would be unhelpful under standard deployment conditions (where multiple independent samples, different temperatures, or diverse models are used). Second, artifact diagnostics can reveal the immediate cause of a null: here the cause was absence of validator detections (repair path never invoked). Operational evaluation should therefore pair estimator choices with task selection that yields nontrivial validator-failure rates when the goal is to measure repair efficacy. For safety-critical deployments, a practical path is to strengthen validator fidelity (e.g., integrate vendor CLI semantics, run candidate changes against a packet-level emulator or staged lab, or add dynamic checks for protocol convergence) so that the validator better approximates the operational harm surface that repairs must address.

Methodological recommendations grounded in archived evidence. The execution and manifest artifacts identify specific, artifact-traceable modifications for follow-on preregistered experiments that would place the repair mechanism under test: (1) relax shared_initial_candidate semantics to generate independent first-pass candidates per condition so that generator variability can produce baseline failures that repairs might fix (this requires changing generation_semantics and re-defining the estimand); (2) increase the expected baseline failure rate via task selection (prefer manifest entries annotated "extreme" or compose adversarial task variants) so that repairs are triggered with nontrivial probability; (3) preregister explicit sampling parameters (temperature sweeps) instead of leaving temperature null/unset; the archived model_configuration.json makes clear that temperature = null in this run and that null/unset does not imply deterministic sampling; (4) augment the validator by integrating higher-fidelity execution backends (vendor CLI emulation or packet-level simulation) to raise construct validity for runtime failure modes; and (5) increase permitted repair iterations or diversify repair strategies so repairs have more opportunity to converge on a valid candidate when initial violations are detected. Each of these recommendations follows directly from the execution artifacts (e.g., stage_counts showing only shared_initial_generation = 200 and the repair_applied = false flags across the sample) and the preregistration sensitivity analysis that assumed nonzero baseline unsafe rates.

Reproducibility and artifact auditing. The artifact bundle contains per-episode validator traces (validation_before/validation_after), raw shared initial responses, the execution manifest, model_configuration.json (declared_model_version = "gpt-5-mini", temperature = null), and analysis/condition_summary.csv and analysis/paired_contingency_table.csv. These artifacts permit independent auditors to verify that: (a) exactly one shared initial generation per pair was produced and reused in both arms (provider audit: shared_initial_generation = 200); (b) validator.valid was true for all episodes and no repair prompts or repair traces were created (repair_attempt_count = 0); and (c) the exact contingency counts driving the McNemar test are reproducible. Importantly, auditors should not infer deterministic model sampling from temperature = null; the execution manifest explicitly records that temperature was unset and that sampling nondeterminism cannot be excluded for the single sampled generation per pair.

Threats to external validity and next steps. The principal external validity limits are single-model scope (declared_model_version = "gpt-5-mini" via the archived adapter), synthetic deterministic task bank, and the validator abstraction described above. These limits are documented in the registered limitations and in the evidence bundle. The most direct path to broader external validity is a controlled factorial follow-on that crosses (i) independent vs shared generation semantics, (ii) model temperature and model family, and (iii) validator fidelity (syntactic/workflow vs emulator-integrated). Such a design would separate generator error rates from repair efficacy and allow estimation of the number-needed-to-repair under realistic sampling variability. All of these modifications are prespecified in the paper's preregistered follow-on recommendations and can be implemented while preserving the same deterministic task manifest structure for continuity of comparison.

Summary. The expanded artifact diagnostics show that the run produced an informative but narrowly scoped result: deterministic validation found no violations for the single sampled candidate per intent, so the permitted guarded repair path was never exercised and could not change outcomes. This clarifies the causal limit of the executed estimand and points directly to preregistered experimental modifications needed to measure repair efficacy under conditions that produce validator failures and thus allow repairs to act.

VI. Limitations

- Shared_initial_candidate semantics: the design produced exactly one sampled initial generation per intent and reused it across baseline and guarded conditions; this measures validator/repair effects only and cannot detect differences caused by independent per-condition sampling or prompt engineering.
- Temperature unset (temperature = null) in the archived model configuration: null/unset is not evidence of deterministic sampling and does not imply temperature = 0; sampling nondeterminism may still have occurred during the single sampled generation.
- Single model and single hosted provider: results reflect gpt-5-mini under the archived adapter; generalization to other models or model families is unsupported by these artifacts.
- Synthetic task bank and validator abstraction: tasks were synthetic 'intent_configuration_repair_v1' items and the deterministic validator is an abstraction; realistic vendor CLI idiosyncrasies, emergent runtime interactions, and hardware effects were not executed on live devices.
- No observed validator failures: all initial candidates were validator-valid, so the experiment could not assess the repair step's effectiveness; the null effect therefore re-

flects the task bank and sampling semantics rather than evidence that GVR is unnecessary in general.

VII. Conclusion

We executed a fully preregistered paired confirmatory experiment that measures the downstream effect of a deterministic validator plus an allowed single automated repair on the exact same sampled LLM candidate per intent (shared_initial_candidate semantics). Using the declared generator gpt-5-mini and deterministic_netops_validator_v5 on 200 paired intents, every shared initial candidate passed validation and no repairs were attempted or applied; guarded and baseline outcomes were identical for the executed estimand (paired difference = 0.0; McNemar p = 1.0; paired bootstrap CI = [0.0, 0.0]). This artifact-grounded null result demonstrates an estimand boundary: under shared_initial_candidate semantics and the executed task bank the guarded pipeline could not be exercised and therefore could not change outcomes. It does not imply that GVR pipelines are unnecessary generally. Preregistered follow-on experiments that (1) remove shared_initial_candidate semantics, (2) increase task difficulty or inject adversarial inputs, (3) set explicit sampling parameters, and (4) raise validator fidelity are required to empirically evaluate repair efficacy under realistic sampling variability and adversarial conditions. The archived artifacts and reconciliation files enable exact reproduction of the executed estimand and provide a secure base for precise preregistered extensions.

References [1] A. Alansari and H. Luqman, "Large Language Models Hallucination: A Comprehensive Survey," arXiv preprint arXiv:2510.06265, 2025. [2] B. Zhang, H. Rao, and D. Zhao, "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents," J. Autom. Cloud-Native Syst., 2024. [3] P. Jana et al., "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback," Proc. ACM Symp. (2026). [4] Y. Miao et al., "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models," arXiv:2309.05557, 2023. [5] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," ACM Comput. Surv., 2025.

One-line reiteration of the primary estimand limit: the preregistered primary estimand intentionally used shared_initial_candidate semantics; evaluating per-condition independent sampling requires a different preregistration and analysis plan (explicitly recommended above).

Disclosure Statement

Disclosure (concise): The human Research Director selected the broad topic family (Generative AI / LLMs for NetOps) before the autonomy lock. After the autonomy lock the autonomous system performed literature review, experiment selection, preregistration (PREREG-CAND-001-VER-GVR-2026-09-01), code generation, execution, deterministic validation, automated analysis, and manuscript drafting without manual human editing of technical content. Declared model used in execution: gpt-5-mini via hosted adapter 'openai_responses' (execution used model_calls_used = 200; archived model_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling and does not imply temperature = 0). Generation semantics executed: shared_initial_candidate — exactly one sampled initial generation per intent was produced and deterministically reused for both baseline and guarded conditions. Validator: deterministic_netops_validator_v5 scored candidates using 'full_intent_constraint_validation' and a single automated repair iteration was permitted in guarded episodes; in this run the validator flagged no violations and no repairs were applied (repair_attempt_count = 0; repair_applied_count = 0). Execution accounting: completed_episode_count = 400 (200 paired intents) completed without preregistration deviations. Master prompt immutable reference: provenance/master_prompt.txt — SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba. Pre-lock infrastructure development and rehearsal occurred but pre-lock artifacts were not used as empirical evidence in this final run. After the autonomy lock no human manually rewrote technical results, manuscript text, figures, or references.

References

[1] Aisha Alansari, Hamzah Luqman, "Large Language Models Hallucination: A Comprehensive Survey", 2025, 10.48550/arxiv.2510.06265

[2] Boning Zhang, Hengning Rao, Derek Zhao , "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents", 2024, 10.69987/jacs.2024.40308

[3] Prithwish Jana, Sam Davidson, Bhavana Bhasker, Andrey Kan, Anoop Deoras, Laurent Callot, "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback", 2026, 10.1145/3786583.3786898

[4] Yukai Miao, Yu Bai, Li Chen, Dan Li, Haifeng Sun, Xizheng Wang, Ziqiu Luo, Ren, Yanyu, Dapeng Sun, Xiuting Xu, Qi Zhang, Chao Xiang, Xinchi Li, "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models", 2023, 10.48550/arxiv.2309.05557

[5] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635